

Islamic University of Technology

Lab: Greedy & Flow-Network Engineering

CSE 4810: Algorithm Engineering
Group 2 — Kruskal's Algorithm & Min-Cut

ASH, ABT(PT)
July 30, 2026

General Instruction

This lab consists of **three tasks** and is designed to fit a **2-hour** session. Written derivations are replaced by **live implementation, profiling, and verbal defense**. For every task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file(s).
- Step 2.** Compile and run the program, feeding it the exact stress-test / verification inputs specified.
- Step 3.** Produce the requested live artifact — a runtime plot, an MST weight, or a verified min-cut — and keep your terminal/IDE open.
- Step 4.** Be ready to defend your engineering choices verbally using the artifact on screen; no written report is collected.

Task 1 — Kruskal's Algorithm (Union-Find)	~35 minutes
Task 2 — Min-Cut (via Max-Flow Min-Cut Theorem)	~55–60 minutes
Task 3 — Live Profiling & Verification	~25–30 minutes

Task 2 is the heaviest task — budget your time accordingly. If you are running short, prioritize a *correct* Min-Cut implementation over a fully-polished Task 3 plot.

Files Provided

<code>KruskalMST.cpp</code>	Kruskal's Algorithm skeleton (DSU-based)
<code>MinCut.cpp</code>	Min-Cut skeleton (max-flow engine + cut extraction)
<code>plotter_group2.py</code>	Plots runtime data from both tasks

Task 1: Kruskal's Algorithm with an Optimized Union-Find

Objective: Build a Minimum Spanning Tree using Kruskal's Algorithm, backed by a Disjoint-Set Union (DSU) with union by rank and path compression.

Component	Strategy	Why It Matters
DSU::find	Path compression — every node visited during a <code>find</code> is re-attached directly to the root	Flattens the tree over repeated calls, driving amortized cost toward $O(\alpha(n))$, effectively constant.
DSU::unite	Union by rank — always attach the shorter tree under the taller tree's root	Prevents the DSU forest from degenerating into a long chain, keeping <code>find</code> paths short even before compression kicks in.

0.1 TODO 1: Implement `DSU::find(x)` with path compression

[Hint] Recursively (or iteratively) follow `parent[]` pointers to the root, then re-point every visited node directly to that root before returning.

0.2 TODO 2: Implement `DSU::unite(x, y)` with union by rank

[Hint] Find the roots of `x` and `y`. If equal, return `false` (would form a cycle). Otherwise attach the lower-rank root under the higher-rank root; if ranks are equal, attach either way and increment the resulting root's rank.

0.3 TODO 3: Implement `kruskal_mst(n, edges)`

[Hint] Sort all edges by weight ascending. Walk the sorted list; call `unite` on each edge's endpoints, and add the edge's weight to your running total only if `unite` returned `true`.

Sanity Check — Expected Output

```

1 --- KRUSKAL'S MST SANITY CHECK (n=5) ---
2 Brute-force reference : 16
3 Kruskal (yours)       : 16 (should match reference)
4 -----

```

A **brute-force reference** (`brute_force_mst`) is already implemented in the skeleton file for validation on the tiny sanity-check graph. Do not modify it — it is exponential-time and only usable for small n .

Compile & Run

```

1 g++ -O2 -std=c++17 -o kruskal KruskalMST.cpp
2 ./kruskal

```

Task 2: Min-Cut from Scratch (Max-Flow Min-Cut Theorem)

Objective: Implement Edmonds-Karp max-flow as the underlying engine, then extract the minimum cut directly from the saturated residual graph.

Key Idea — Max-Flow Min-Cut Theorem

Once max-flow has fully saturated a network, let S be the set of vertices still reachable from the source in the *residual* graph. The min cut is exactly the set of original edges crossing from S to $V \setminus S$, and its total capacity **always equals** the max-flow value — this is not a coincidence, it is a theorem, and your code should demonstrate it numerically.

0.4 TODO 1: Implement `bfs_augmenting_path(source, sink, parentEdge)`

[Hint] Identical requirement to the Max-Flow task: BFS over edges with positive residual capacity ($\text{capacity} - \text{flow} > 0$), recording `parentEdge[]` for path reconstruction.

0.5 TODO 2: Implement `max_flow(source, sink)`

[Hint] Identical requirement to the Max-Flow task: repeatedly find an augmenting path, compute its bottleneck, and push that much flow along every edge (and subtract it from every reverse edge) until no augmenting path remains.

0.6 TODO 3: Implement `find_min_cut(source)`

[Hint] Call this only after `max_flow` has terminated. Run a BFS/DFS from `source` using only edges with positive residual capacity to find S . Then scan all original edges ($\text{capacity} > 0$) and collect every one whose tail is in S and whose head is not — that is your min cut.

Sanity Check — Expected Output

```
1 --- MIN-CUT SANITY CHECK (6 vertices) ---
2 Max-Flow value      : 23 (Expected 23)
3 Min-Cut edges:
4   1 -> 3 (capacity 12)
5   4 -> 3 (capacity 7)
6   4 -> 5 (capacity 4)
7 Total Min-Cut capacity: 23 (should equal Max-Flow value = 23)
8 -----
```

Compile & Run

```
1 g++ -O2 -std=c++17 -o mincut MinCut.cpp
2 ./mincut
```

Task 3: Live Profiling & Min-Cut Verification

Objective: Quantify Kruskal's runtime growth, and empirically confirm the Max-Flow Min-Cut theorem across multiple random networks.

Runs to Perform

- **Kruskal's Density Sweep:** `run_benchmark()` in `KruskalMST.cpp` times your implementation across increasing n at two densities (sparse ≈ 0.02 , dense ≈ 0.2). Run it and inspect `data_kruskal.txt`.
- **Plot it:** `python3 plotter_group2.py data_kruskal.txt data_mincut.txt` produces `kruskal_runtime.png` and `mincut_runtime.png`.
- **Theorem Verification:** `verify_maxflow_mincut_equivalence()` in `MinCut.cpp` runs your max-flow and min-cut on two random networks ($n = 15$ and $n = 30$) and reports whether the values match. Both must report YES.

Core Engineering Defense (Verbal Check)

- **Amortized Cost:** Point to your Kruskal's runtime plot and explain, in terms of path compression and union by rank together (not just one of them), why `find` calls stay cheap even after thousands of unions.
- **Why S , not $V \setminus S$:** Explain why the min cut is defined using vertices reachable *from the source* in the residual graph, and what would go wrong if you instead used vertices that can reach the *sink*.
- **Theorem, Not Coincidence:** Using your `verify_maxflow_mincut_equivalence()` output, explain in your own words why the max-flow value and min-cut capacity are provably equal for *any* network, not just the ones you tested.

The instructor may ask you to identify, from your Task 2 sanity-check min-cut, which single edge's capacity increase would raise the network's max-flow value, and which edges' capacity increases would have *no effect* at all. Justify your answer without re-running the code.